



Módulo III | Clase 8

Transformación de datos y evaluación de distribuciones





¿Ponemos a grabar el
taller?

Repaso: ¿dónde quedamos?



- Feature engineering: seleccionar, combinar y transformar variables
 - Obtención de variables derivadas
 - Encoding de categóricas: `LabelEncoder()`, `get_dummies()`, `OneHotEncoder()`
-
- Binning: agrupar una variable continua en rangos
 - Re-escalado al rango $[0,1]$ con `MinMaxScaler()`
 - Hoy seguimos con lo que falta: la escala y la forma de las variables numéricas

¿Qué vamos a ver hoy?



- Evaluación de distribuciones
- Escalado de variables
- Transformaciones de la forma

- Buenas prácticas
- Práctica: Notebook 12



Evaluación de distribuciones

¿Por qué mirar la distribución primero?

Muchos algoritmos asumen normalidad: LDA, Naive Bayes gaussiano y los supuestos sobre los residuos en regresión lineal.

Los que se basan en distancias (KNN, K-means, SVM) se dejan dominar por las variables de mayor rango.

Los que optimizan por descenso por gradiente (redes neuronales, regresión logística) convergen mucho más lento con escalas dispares.

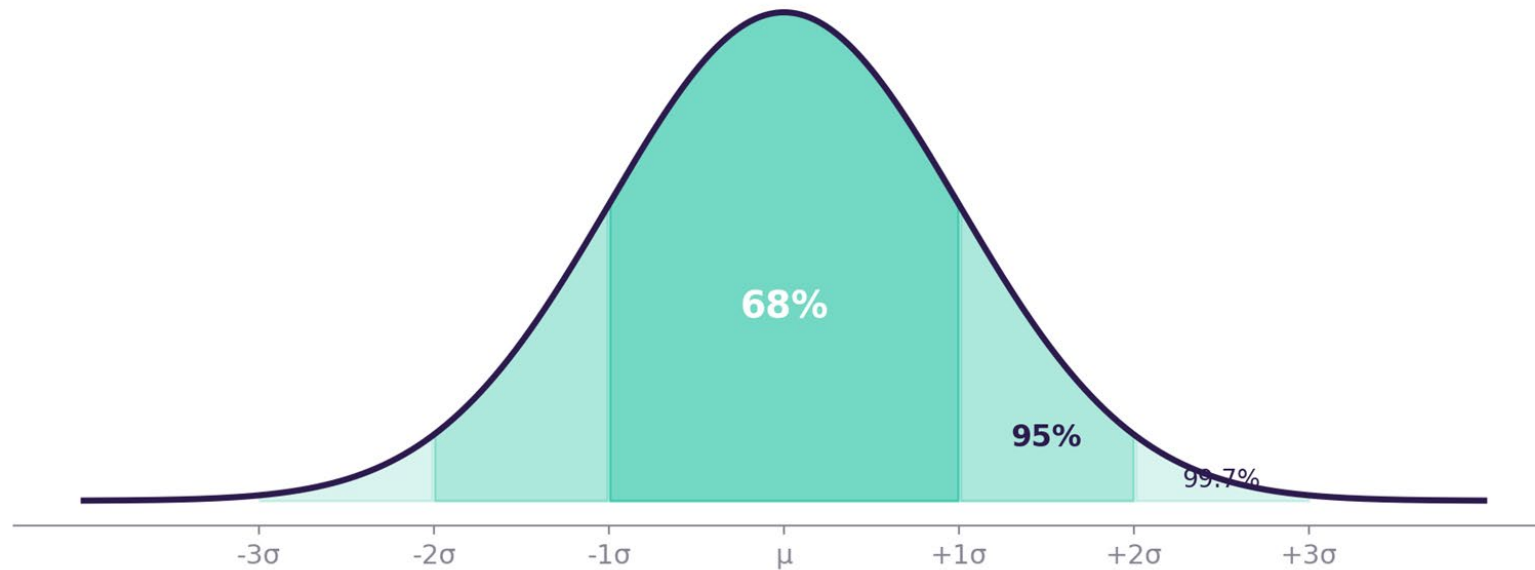
En una variable muy asimétrica, la media y el desvío dejan de ser buenos resúmenes.



Todo esto tendrá sentido poco a poco a medida que avancemos con ML

La distribución normal como referencia

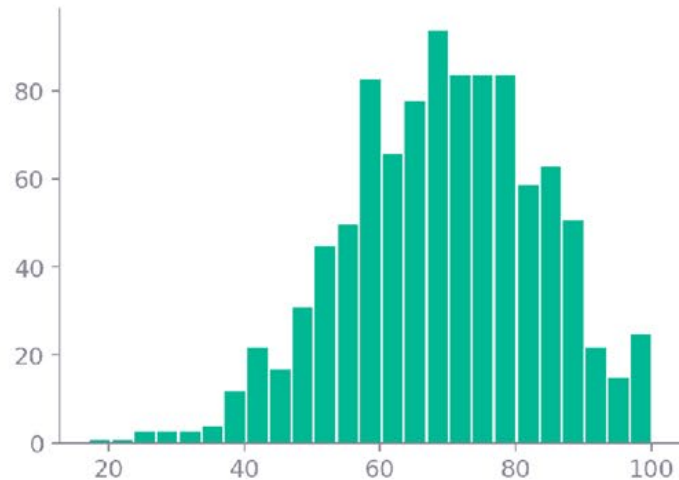
Simétrica alrededor de la media · Media = mediana = moda · Es la referencia contra la que comparamos, no un requisito: no toda variable tiene que ser normal



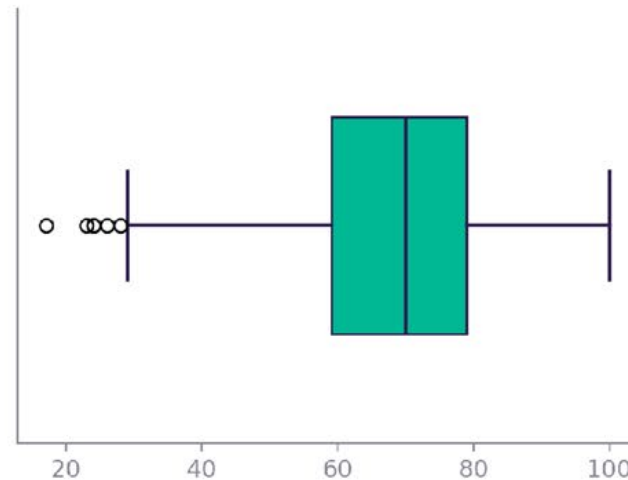
Cómo evaluamos: métodos gráficos

Histograma: la forma general · Boxplot: mediana, dispersión y outliers · Q-Q plot: cuantiles observados contra los teóricos de una normal

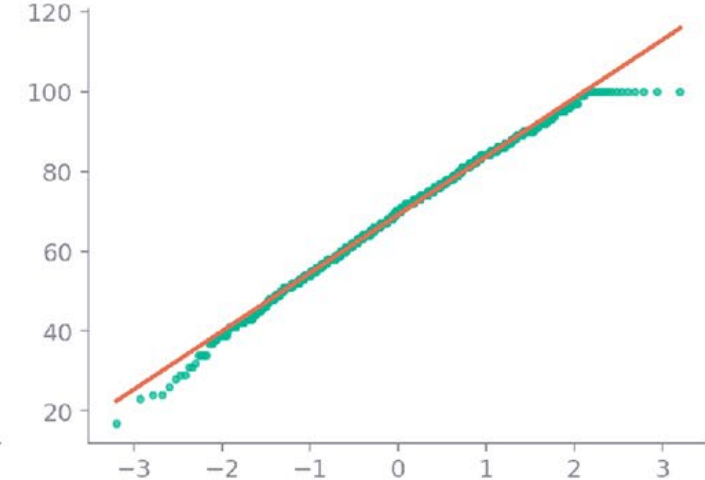
Histograma



Boxplot



Q-Q plot



● Cómo evaluamos: medidas numéricas

- Asimetría (skewness) — `.skew()`
- $\approx 0 \rightarrow$ simétrica
- $> 0 \rightarrow$ cola hacia la derecha (ingresos, precios)
- $< 0 \rightarrow$ cola hacia la izquierda
- $|\text{skew}| < 0.5$ es aceptable · $|\text{skew}| > 1$ pide transformación



- Curtosis (kurtosis) — `.kurt()`
- Cuán pesadas son las colas respecto de una normal
- Valores altos anticipan outliers

Cómo evaluamos: tests de normalidad

Shapiro-Wilk: `stats.shapiro()`, para $n < 5000$

D'Agostino-Pearson: `stats.normaltest()`, para n mediano o grande

Kolmogorov-Smirnov: `stats.kstest()`, contra cualquier distribución

H_0 : los datos provienen de una distribución normal. Si $p < 0.05$, la rechazamos.

Cuidado con n grande: cualquier desviación mínima da significativa y el test rechaza casi siempre. En el notebook las tres variables dan “no normal”, con asimetría de -0.28 y Q-Q plots perfectos. El test no manda solo.

Del diagnóstico a la decisión

Escalas distintas, forma razonable → StandardScaler o MinMaxScaler

Escalas distintas y outliers marcados → RobustScaler

Asimetría fuerte ($|\text{skew}| > 1$) → logaritmo, Box-Cox o Yeo-Johnson

Distribución que no cede con nada → QuantileTransformer

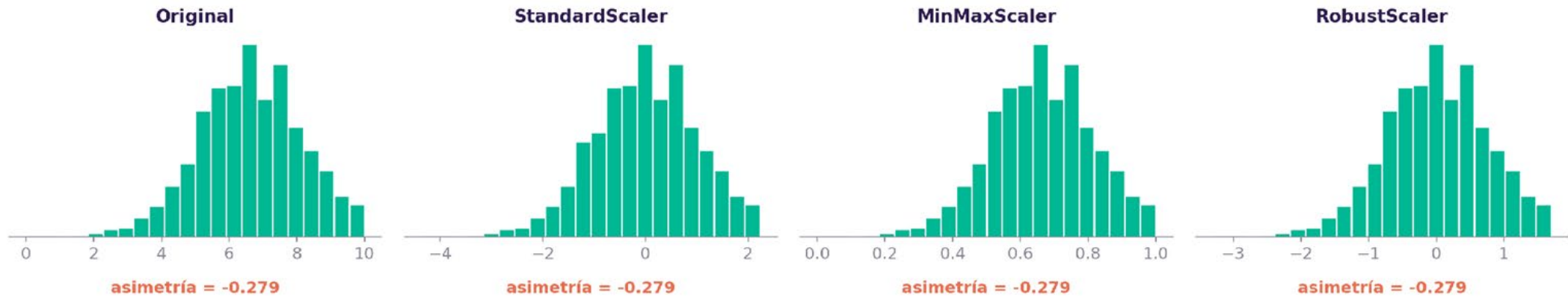
Árboles, random forest o boosting → no hace falta escalar



Escalado de variables

Escalar no cambia la forma

Escalar mueve el centro y cambia la unidad del eje. La asimetría es idéntica en las cuatro versiones: si la variable está sesgada, sigue sesgada



Estandarización — StandardScaler()

Deja la variable con media 0 y desvío estándar 1.

Es el mismo z-score que usamos en la clase 6 para detectar outliers.



No acota el rango, y es sensible a outliers: μ y σ se ven arrastrados por ellos

- `.StandardScaler()`

$$Z_i = \frac{x_i - \mu}{\sigma}$$

Support vector machines (SVM), regresión logística, redes neuronales y PCA

Re-escalado — MinMaxScaler()

Lleva todos los valores al rango
[0, 1].

Ya lo vimos en la clase 7: lo
retomamos para compararlo.



Preserva la forma, pero es muy sensible a
outliers: un valor extremo comprime al resto
contra el 0

- .MinMaxScaler()

$$x' = \frac{x - x_{mean}}{x_{max} - x_{min}}$$

Escalado robusto — RobustScaler()

Usa mediana y rango intercuartílico (RIC) en vez de media y desvío.

Es la respuesta directa a los outliers de la clase 6.



La mediana y el RIC casi no se mueven ante valores extremos

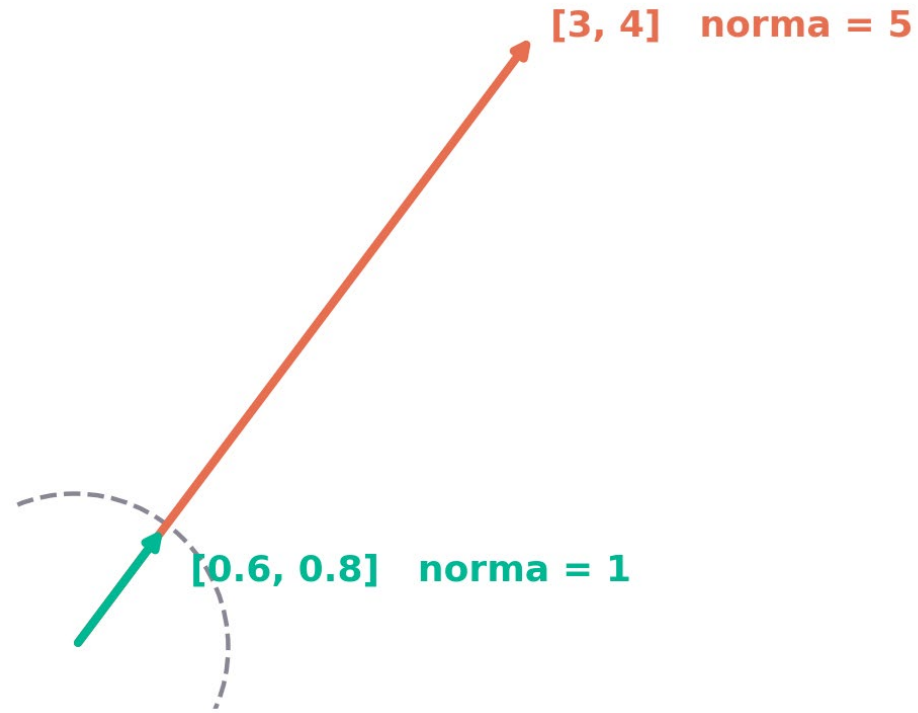
- .RobustScaler()

$$X_i = \frac{x_i - \text{mediana}}{RIC}$$

Cuando los outliers son información válida y no un error de carga

Normalizer(): cuidado con el nombre

No vuelve normal una distribución ni lleva los valores a $[0,1]$. Opera por fila: escala cada observación a norma 1. Aplicado a una sola columna, todas las filas dan 1.0



Comparación

Estandarización · media 0, desvío 1 · rango libre · sensible a outliers · `StandardScaler()`

Re-escalado · lleva al rango [0,1] · muy sensible a outliers · `MinMaxScaler()`

Escalado robusto · mediana 0, escala RIC · no sensible a outliers · `RobustScaler()`

Normalización L2 · norma 1 por fila · `Normalizer()`

Los tres primeros trabajan por columna. El cuarto, por fila.



Descanso

Nos vemos en 10 minutos



Transformar la forma de la distribución

Logaritmo y raíz

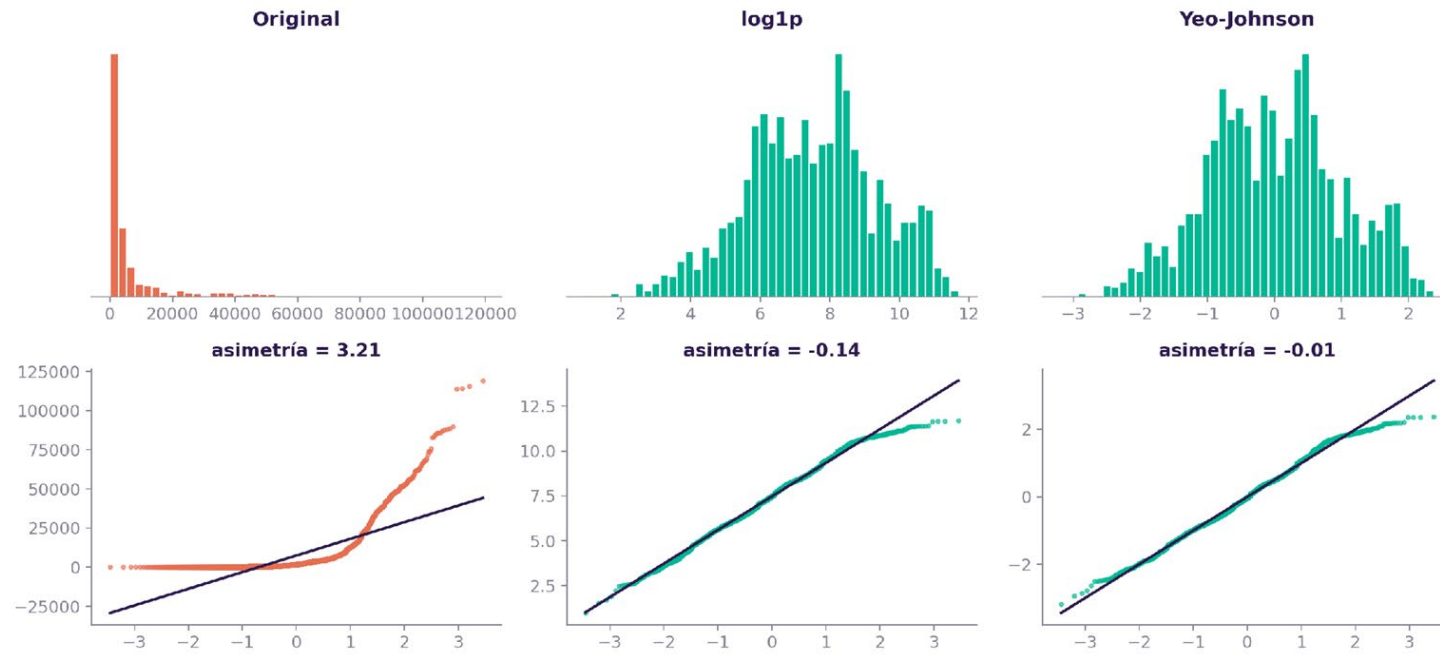
- Logaritmo — `np.log1p(x)`
- Calcula $\log(1+x)$, así que tolera ceros
- Comprime fuertemente los valores grandes
- Requiere valores ≥ 0



- Raíz cuadrada — `np.sqrt(x)`
- Efecto más suave, sirve para conteos
- Ojo: el modelo predice en la escala transformada. Para interpretar, `np.expm1()`

PowerTransformer(): Box-Cox y Yeo-Johnson

Buscan el exponente λ que más acerca la variable a una normal. Box-Cox pide valores positivos; Yeo-Johnson acepta ceros y negativos. Sobre el PBI per cápita: de 3.21 a -0.01





Buenas prácticas

fit en train, transform en test

El scaler aprende parámetros de los datos (μ , σ , mín, máx, mediana, RIC).

Si los aprende del dataset completo, información del test se filtra al entrenamiento: es data leakage, y las métricas quedan infladas.

Correcto: `X_train_esc = scaler.fit_transform(X_train)`
`X_test_esc = scaler.transform(X_test)`

Incorrecto: `X_esc = scaler.fit_transform(X)` ← antes de separar

Regla: `fit_transform` una sola vez, sobre train. Sobre todo lo demás, `transform`.

Qué escalar, qué no, y para qué modelos



- Escalamos: variables numéricas continuas con escalas o unidades distintas
- No lo necesitan: árboles de decisión, random forest, gradient boosting
- Necesitan escalado: KNN, K-means, SVM, regresión logística, redes neuronales, PCA
- No escalamos: dummies y binarias, ordinales donde el orden importa, la variable objetivo

Estandarización y escalado con IA

La IA es útil para decidir, no para elegir por nosotras.

Pedirle que compare opciones: “tengo una variable de ingresos con asimetría 2.3 y outliers, ¿qué scaler conviene y por qué?”

Pedirle que explique un resultado: por qué el Q-Q plot se curva en los extremos.

Pedirle el código base y revisar que respete fit en train y transform en test.

Siempre verificar: los asistentes confunden normalización, estandarización y `Normalizer()`. Si dice que `Normalizer()` vuelve normal una variable, está mal.



Sección práctica:

Aplicamos lo visto con la
Notebook 12

En la sala general

Notebook 12 — partes 0 a 5

Repaso, evaluación de distribuciones, los tres scalers, qué hace realmente Normalizer(), transformaciones de la forma y separación train/test.

20 minutos



Trabajamos en salas

Notebook 12, Parte 6
Diagnóstico y transformación de
variables

Puesta en común

Cada grupo cuenta:

Qué variables eligió y qué encontró en el diagnóstico

Qué transformación aplicó y con qué criterio

Si mejoró o no, y qué explicación le encuentra

Lo importante no es el código: es la justificación de cada decisión.

20 minutos





¿Alguna consulta?

DESAFÍO 10



Para la siguiente clase, sobre su tema y dataset:

Evaluar la distribución de las variables numéricas
(histograma, asimetría, Q-Q plot)

Aplicar el escalado que corresponda

Transformar las que tengan asimetría marcada

Justificar por qué eligieron cada transformación



FUNDACIÓN
YPF

¡Muchas Gracias!

En la próxima clase haremos un repaso práctico general de todo lo visto en el módulo

